

POLITECNICO
MILANO 1863

Best Bike Paths, Luca Suzani, Alberto
Spezzapria, Alessio Pesavento

# Design Document

| | |
|---|---|
| **Deliverable:** | DD |
| **Title:** | Design Document |
| **Authors:** | Luca Suzani, Alberto Spezzapria, Alessio Pesavento |
| **Version:** | 1.0 |
| **Date:** | 07-January-2026 |
| **Download page:** | https://github.com/Alessio01103/PesaventoSpezzapriaSuzani.git |
| **Copyright:** | Copyright © 2025, Luca Suzani, Alberto Spezzapria, Alessio Pesavento – All rights reserved |

# Contents


Table of Contents . . . 3
List of Figures . . . 5
List of Tables . . . 5

**1 Introduction** **7**
1.1 Purpose . . . 7
1.2 Scope . . . 7
1.3 Definitions, Acronyms, and Abbreviations . . . 8
1.3.1 Definitions . . . 8
1.3.2 Acronyms . . . 9
1.3.3 Abbreviations . . . 9
1.4 Revision History . . . 9
1.5 Reference Documents . . . 9
1.6 Document Structure . . . 10
1.7 Use of Generative AI Tools . . . 11

**2 Architectural Design** **13**
2.1 Overview . . . 13
2.1.1 High-Level Architecture Description . . . 13
2.1.2 Security Model: Row Level Security (RLS) . . . 14
2.2 Component view . . . 14
2.2.1 Hybrid Client Components . . . 14
2.2.2 Managed Cloud Components (BaaS) . . . 14
2.2.3 Edge Functions . . . 15
2.3 Deployment view . . . 16
2.3.1 Network Communication . . . 16
2.4 Runtime view . . . 17
2.5 Component interfaces . . . 23
2.5.1 Hardware Abstraction Interfaces . . . 23
2.5.2 Cloud Network Interfaces (BaaS API) . . . 24
2.5.3 External Interfaces . . . 26
2.6 Selected architectural styles and patterns . . . 26
2.6.1 Architectural Styles . . . 26
2.6.2 Design Patterns . . . 27
2.7 Other design decisions . . . 28
2.7.1 Adoption of Hybrid Mobile Architecture . . . 28
2.7.2 Usage of PostgreSQL with PostGIS . . . 28
2.7.3 Centralized Logic for Merging and Ranking . . . 28
2.7.4 Direct-to-Database Communication Pattern . . . 28

**3 User Interface Design** **29**

**4 Requirements Traceability** **32**
4.1 Requirements Traceability . . . . . 32
4.1.1 User Account Management . . . . . 32
4.1.2 Activity Recording . . . . . 33
4.1.3 Path Creation (Automatic) . . . . . 33
4.1.4 Path Creation (Manual) . . . . . 33
4.1.5 Search and Ranking . . . . . 34
4.1.6 Path Management . . . . . 34

**5 Implementation, Integration and Test Plan** **35**
5.1 Implementation, Integration & Testing . . . . . 35
5.1.1 Overview . . . . . 35
5.1.2 Implementation Plan . . . . . 35
5.1.3 Component Integration . . . . . 37
5.1.4 Testing Strategy . . . . . 37

**6 Effort Spent** **38**

Copyright © 2025, Luca Suzani, Alberto Spezzapria, Alessio Pesavento – All rights reserved 

# List of Figures


2.1 [UC1] User Registration Sequence Diagram . . . . . 17
2.2 [UC2] System Access Sequence Diagram . . . . . 18
2.3 [UC3] Activity Recording Sequence Diagram . . . . . 19
2.4 [UC4] Creation of a manual path Sequence Diagram . . . . . 20
2.5 [UC5] Publication of a path Sequence Diagram . . . . . 21
2.6 [UC6] Path Feedback reporting Sequence Diagram . . . . . 22
2.7 [UC7] Search for a path Sequence Diagram . . . . . 23

3.1 Main dashboard summarizing service status, recent notifications, and quick actions. . . . 29
3.2 Trip recording interface with live metrics and waypoint controls. . . . . 30
3.3 Explore curated and suggested routes with filters and key stats. . . . . 30
3.4 List of saved routes with quick access to details and navigation. . . . . 31
3.5 Settings grouped by account, privacy, and notifications. . . . . 31

# List of Tables


2.1 IGeolocationProvider Interface . . . . . 24
2.2 IMotionSensor Interface . . . . . 24
2.3 IAuthService Interface . . . . . 25
2.4 ISpatialDataStore Interface . . . . . 25
2.5 ISearchEngine Interface . . . . . 26
2.6 IWeatherProvider Interface . . . . . 26

4.1 Traceability for User Account Management . . . . . 32
4.2 Traceability for Activity Recording . . . . . 33
4.3 Traceability for Automatic Path Creation . . . . . 33
4.4 Traceability for Manual Path Creation . . . . . 33
4.5 Traceability for Search and Ranking . . . . . 34
4.6 Traceability for Path Management . . . . . 34

6.1 Hours of work spent by each author redacting the DD document . . . . . 38

# Chapter 1

# Introduction

## 1.1 Purpose

The purpose of this document is to provide a comprehensive architectural design for the **Best Bike Paths (BBP)** system. It serves as the primary bridge between the requirements analysis phase, detailed in the *Requirements Analysis and Specification Document (RASD)*, and the implementation phase.

This document is intended for developers, project managers and testers involved in the project. Its specific goals are to:

- Define the **High-Level Architecture**, identifying the subsystems and their distribution.
- Detail the **Low-Level Components**, specifying their responsibilities, internal logic, and interfaces.
- Justify the **Design Decisions** and architectural styles adopted.
- Map the functional requirements to specific software modules to ensure full **Traceability**.

By following this design, it will be possible to implement a scalable, reliable and consistent system that fulfills the user needs regarding cycling safety and path discovery.

## 1.2 Scope

The **Best Bike Paths (BBP)** system is a crowdsourcing platform designed to collect, store, search and rank cycling itineraries based on user-generated content and sensor telemetry. The application aims to solve the fragmentation of cycling data by merging individual user tracks into reliable, high-quality public paths.

The scope of the software design covers two main areas:

1. **The Hybrid Client Application:** A unified application for Android, iOS, and Web. It is responsible for:
    - **Telemetry Acquisition:**Interfacing with device hardware (GPS, Accelerometer) via a Native Bridge to track routes and detect road irregularities (e.g., potholes) even in background mode.
    - **Offline Resilience:** Ensuring data integrity and functionality in areas with poor network coverage through local buffering and synchronization mechanisms.
2. **The Serverless Cloud Infrastructure:** A managed backend environment responsible for:
    - **Data Consolidation (Merging):** Executing geospatial algorithms to merge conflicting data from multiple users into a "Single Source of Truth" for each path.

- **Dynamic Ranking:** Calculating a suitability score for paths based on safety, road condition, and user feedback freshness.
- **Security:** Enforcing data isolation and access control via Row Level Security (RLS).

The system interacts with external services, specifically third-party **Weather APIs**, to enrich path metadata. The design excludes the implementation of the physical cycling infrastructure itself, focusing solely on the digital platform to map and evaluate it.

## 1.3 Definitions, Acronyms, and Abbreviations

### 1.3.1 Definitions

- **Backend-as-a-Service (BaaS)**: A cloud computing model adopted in this architecture where backend responsibilities (database, authentication, hosting) are outsourced to a managed provider (Supabase), allowing development to focus on frontend logic.
- **Cron Job**: A time-based job scheduler in Unix-like computer operating systems, used in this project to trigger periodic maintenance tasks (e.g., the decay of old feedbacks) on the server.
- **Edge Function**: A piece of serverless application logic (e.g., the path merging algorithm) that runs on the cloud provider's infrastructure, triggered by specific database events or HTTP requests.
- **Hybrid Application**: The chosen architectural approach where the mobile app is built using web technologies (React/TypeScript) but executes within a native container to access device hardware.
- **Kalman Filter**: An algorithm that uses a series of measurements observed over time (containing statistical noise) to produce estimates of unknown variables (e.g., precise GPS coordinates) that tend to be more accurate than those based on a single measurement alone.
- **Native Bridge**: The software mechanism (provided by Capacitor) that allows the web-based client code to communicate with the native operating system and access sensors like the Accelerometer and GPS in the background.
- **Object Storage**: A computer data storage architecture that manages data as distinct units (objects), used in this system to store large binary assets like user profile pictures.
- **PostGIS**: An open-source software extender for the PostgreSQL database, used in this project to store, index, and query complex geospatial data (geometries) efficiently.
- **Row Level Security (RLS)**: A security mechanism implemented directly in the database engine that restricts access to specific data rows based on the user's authentication token, eliminating the need for application-level security filters.
- **Thick Client**: An architectural style where the client application (running on the user's device) handles the bulk of data processing, validation, and temporary storage, contrasting with a Thin Client that relies heavily on the server.
- **Webhook**: A mechanism for an application to provide other applications with real-time information. In this system, database webhooks trigger Edge Functions automatically when data is inserted or updated.

Copyright © 2025, Luca Suzani, Alberto Spezzapria, Alessio Pesavento – All rights reserved 

### 1.3.2 Acronyms

- **API**: Application Programming Interface.
- **BaaS**: Backend-as-a-Service.
- **CPU**: Central Processing Unit.
- **CRUD**: Create, Read, Update, Delete.
- **JSON**: JavaScript Object Notation.
- **JWT**: JSON Web Token (used for stateless authentication).
- **KPI**: Key Performance Indicator.
- **OS**: Operating System.
- **PWA**: Progressive Web Application.
- **RLS**: Row Level Security.
- **RPC**: Remote Procedure Call (used to invoke Edge Functions).
- **SDK**: Software Development Kit.
- **SPA**: Single Page Application.
- **SQL**: Structured Query Language.
- **WSS**: WebSocket Secure (used for real-time updates).

### 1.3.3 Abbreviations

- **DB**: DataBase.
- **UI**: User Interface.
- **VM**: Virtual Machine.

## 1.4 Revision History

Version 1.0: 07/01/2025

## 1.5 Reference Documents

- **[1] RDO**: Assignment RDD AY 2025-2026 - Software Engineering 2 Project Assignment, Politecnico di Milano.
- **[2] RASD**: Requirement Analysis and Specification Document for “Best Bike Paths”, Version 1.0.
- **[3] Capacitor Documentation**: Official documentation for the Capacitor native runtime. Available at: `https://capacitorjs.com/docs/`
- **[4] TypeScript Language**: Documentation for the TypeScript programming language. Available at: `https://www.typescriptlang.org/`

Copyright © 2025, Luca Suzani, Alberto Spezzapria, Alessio Pesavento – All rights reserved 

- **[5] Supabase**: Documentation for the open-source Firebase alternative (Backend-as-a-Service). Available at: `https://supabase.com/`
- **[6] PostGIS**: Spatial and Geographic objects for PostgreSQL. Available at: `https://postgis.net/`
- **[7] Deno**: Documentation for the Deno runtime (used by Supabase Edge Functions). Available at: `https://deno.com/`
- **[8] Zod**: TypeScript-first schema declaration and validation library. Available at: `https://zod.dev/`
- **[9] React**: The library for web and native user interfaces. Available at: `https://react.dev/`
- **[10] PostgreSQL**: The World's Most Advanced Open Source Relational Database. Available at: `https://www.postgresql.org/`
- **[11] OpenWeatherMap API**: Weather data and forecast API. Available at: `https://openweathermap.org/api`
- **[12] Lovable**: AI-powered full-stack prototyping tool (used for UI Mockups). Available at: `https://lovable.dev/`

## 1.6 Document Structure

The following list provides a brief overview of the contents of each chapter in this document:

- **Chapter 1 Introduction:** Provides an overview of the document's purpose and scope, along with definitions of terms and acronyms used throughout the text. It establishes the context for the architectural decisions.
- **Chapter 2 Architectural Design:** Constitutes the core of this document. It describes the high-level architecture (Hybrid Client + Serverless Cloud), details the internal software components, their interfaces, and the deployment infrastructure. It also justifies the selected architectural styles and design patterns.
- **Chapter 3 User Interface Design:** Presents the visual design of the application through mockups and navigation flow diagrams, illustrating how users interact with the system to perform key tasks.
- **Chapter 4 Requirements Traceability:** Provides a mapping between the functional requirements defined in the RASD and the architectural components described in Chapter 2, ensuring that every requirement is addressed by the software design.
- **Chapter 5 Implementation, Integration, and Test Plan:** Outlines the strategy for developing the system components, the order of integration, and the testing procedures required to verify the system's correctness and reliability.
- **Chapter 6 Effort Spent:** Reports the time spent by each member of the group to contribute to the realization of this document.

Copyright © 2025, Luca Suzani, Alberto Spezzapria, Alessio Pesavento – All rights reserved 

## 1.7 Use of Generative AI Tools

In compliance with the course guidelines regarding academic integrity and content responsibility, we declare that we have used Generative Artificial Intelligence (GenAI) tools to support the drafting of this document (Design Document - DD).

We emphasize that the entire content of this document is the result of our original reasoning; AI was not used to generate architectural decisions or core logic from scratch, but exclusively as a tool for linguistic support, formal verification, and visual prototyping. The details of its usage are reported below:

1. **Tools Used**
    - **Textual Support:** ChatGPT (model GPT-3.5/4o mini), Gemini Advanced (Gemini 3).
    - **Visual Generation (UI Mockups):** Lovable.
2. **Provided Inputs (Prompts and Context)** The inputs provided to the system were exclusively textual and based on preliminary work carried out by the group. Specifically:
    - **For linguistic revision:** We input paragraphs or entire sections written by us in Italian/English, with the request to improve grammatical form, academic vocabulary, and text fluidity, without altering the technical meaning.

        **Example A - For stylistic improvement:** *"Act as a professional Technical Writer reviewing a Software Design Document. I will provide you with a raw text describing the 'Path Merging Algorithm'. Please rewrite it to improve grammatical correctness and use formal academic English suitable for a system architecture description. Do NOT add new information or change the logical flow. Just polish the language."*
    - **For completeness verification:** We provided lists of component interfaces or sequence diagram steps developed by the group, asking the model to flag potential logical ambiguities or missing edge cases.

        **Example B - For completeness verification:** *"Act as a Senior Software Engineer. Below is the list of APIs we defined for the 'Sensor Controller'. Analyze them for consistency with the REST standard. Specifically, identify if there are any missing response codes or parameters that a standard Design Document should include."*
    - **For UI Prototyping (via Lovable):** We provided detailed descriptions of the application screens, including layout structure, color palettes, and required UI elements, to generate high-fidelity visual mockups for Chapter 3.

        **Example C - For UI Generation:** *"Create a mobile screen for a cycling app in a modern style. It must include a map view showing a route, a bottom sheet showing real-time speed (25 km/h) and distance (12.4 km), and a large red circular button to 'Report Hazard'."*
3. **Obtained Outputs**
    - Suggestions for sentence reformulation to make the text more formal and concise.
    - Checklists to verify consistency between different diagrams and textual descriptions.
    - Graphical assets and UI Mockups representing the application interface, which were subsequently refined to match the UX flow.
4. **Verification, Refinement, and Integration** Every output generated by the AI was subjected to a manual critical review by the group members.

Copyright © 2025, Luca Suzani, Alberto Spezzapria, Alessio Pesavento – All rights reserved 

- We did not passively accept the proposed reformulations; instead, we selected and adapted them to ensure they exactly reflected our project specifications.
- The UI mockups generated by Lovable were manually reviewed and used as a basis to define the final interface design, ensuring consistency with the functional requirements.

Copyright © 2025, Luca Suzani, Alberto Spezzapria, Alessio Pesavento – All rights reserved 

# Chapter 2

# Architectural Design

## 2.1 Overview

The **Best Bike Paths (BBP)** system is architected as a **Hybrid Mobile Application** supported by a **Serverless Backend-as-a-Service (BaaS)** infrastructure.

This design strategy allows for a unified codebase (using web technologies like TypeScript/React) to be deployed across both Web and Mobile platforms. Crucially, by leveraging a native runtime bridge (e.g., Capacitor), the application overcomes the limitations of standard web apps. This bridge grants access to native device features, specifically background geolocation and high-frequency motion sensors, which are required for the road Quality Detection and reliability requirements defined in the RASD.

Unlike traditional N-Tier architectures, BBP leverages a thick client model. The client applications contain a significant amount of application logic, communicating directly with the managed cloud database via secure APIs. Complex business logic requiring data aggregation or external integrations is offloaded to edge functions, which execute on-demand in the cloud.

### 2.1.1 High-Level Architecture Description

The system is composed of two main logical nodes:

1. **Hybrid Client Tier:** This tier includes the unified application running on user devices.
    - **Mobile Deployment (Android/iOS):** The application runs within a native container using Capacitor. It accesses hardware sensors via native plugins to perform precise activity tracking and road irregularity detection even when the device screen is off.
    - **Web Deployment:** The same codebase serves as a standard Progressiva Web App for path planning and profile management.
    - **Responsibilities:**
        - Acquisition and pre-processing of raw sensor data (GPS, Accelerometer) via the Native Bridge.
        - Local data caching (Offline-First) to ensure reliability.
        - Direct communication with the BaaS provider for CRUD operations.
2. **Serverless Cloud Tier:** Hosted on a BaaS platform (e.g., Supabase), acting as the scalable backend. It provides:
    - **Data Persistence:** A PostgreSQL database extended with PostGIS for spatial operations.
    - **Authentication:** A built-in identity provider handling OAuth2 and JWT generation.
    - **Edge Logic:** Stateless serverless functions that handle background tasks such as path merging, ranking updates, and weather data enrichment.

### 2.1.2 Security Model: Row Level Security (RLS)

Since clients communicate directly with the database, security is enforced at the database layer using Row Level Security (RLS) policies. Upon login, the Auth Service issues a JSON Web Token (JWT) to the client, which is included in every database query. The database engine evaluates RLS policies for each row to determine if the operation is permitted. This ensures that "direct access" does not imply "unrestricted access," maintaining strict data isolation and integrity.

## 2.2 Component view

This section details the software components within the Client and Cloud tiers, defining their specific responsibilities and interactions.

### 2.2.1 Hybrid Client Components

- **Native Bridge (Capacitor):** The architectural boundary between the web code and the native OS. It exposes a unified API to access hardware features. It ensures that the **Geolocation** and **Accelerometer** continue to operate reliably even when the application is in the background, bridging the gap between the web runtime and the device hardware.
- **Sensor Controller & Filter:** Responsible for logic related to data acquisition.
  - It interfaces with the **Native Bridge** to subscribe to GPS and Accelerometer streams.
  - It implements a **Kalman Filter** to smooth raw GPS coordinates.
  - It analyzes accelerometer data to detect vertical shocks (road surface irregularities like potholes) and buffers this data locally during activity recording.
- **Local Persistence Manager:** Implements the "Offline-First" capability. It uses an embedded database (e.g., SQLite via Capacitor plugin) to store activity traces and pending reports when the network is unavailable. It includes a **Sync Engine** that monitors network reachability and uploads buffered data to the cloud once connectivity is restored.
- **BaaS Client SDK:** The abstraction layer provided by the BaaS platform. It manages the persistent WebSocket connection for real-time updates, handles secure API calls to the database, and automatically refreshes authentication tokens.
- **Map & UI Controller:** Manages the visualization of map tiles and path geometries. It handles user inputs (forms, search queries) and renders the visual feedback for the navigation system.

### 2.2.2 Managed Cloud Components (BaaS)

- **Auth Service (Identity Provider):** Manages user lifecycles (sign-up, login, password recovery). It integrates with third-party OAuth providers (e.g., Google, Facebook) and issues secure JWTs containing user roles and IDs.
- **Spatial Database (PostgreSQL + PostGIS):** The central data repository. It is an active component that:
  - Enforces **RLS Policies** to secure data access.
  - Executes complex **Spatial Queries** (e.g., `ST_DWithin`, `ST_Intersects`) natively, allowing clients to efficiently retrieve paths within a specific viewport.
  - Triggers database events (Webhooks) to activate Edge Functions upon data changes.

Copyright © 2025, Luca Suzani, Alberto Spezzapria, Alessio Pesavento – All rights reserved 

- **Object Storage:** A specialized file storage system for large binary assets, such as user profile pictures or images attached to path reports.

### 2.2.3 Edge Functions

These are stateless functions triggered by database events or scheduled cron jobs:

- `func_path_merging`
  - **Trigger:** Database event (INSERT on `Activities` or `Feedbacks` table).
  - **Responsibility:** It implements the **Data Consolidation & Consensus Logic**. When new data (an activity trace or a manual report) is associated with a path, this function aggregates it with existing reports to determine the path's `CurrentCondition`. It applies a **Majority Voting Algorithm** weighted by data freshness (e.g., resolving a conflict where 3 recent reports say "Good" and 1 says "Bad" by marking the segment as "Good") to ensure the status reflects the actual road situation.
- `func_path_search`
  - **Trigger:** Direct Client Request.
  - **Responsibility:** It implements the **Search & Ranking Algorithm** defined in the RASD. When a user queries for a route (specifying Origin and Destination), this function:
    - Performs a spatial query to find candidate paths connecting the points.
    - Dynamically calculates a **Suitability Score** for each candidate. This score combines the **Geospatial match** (how well it fits the requested Start/End points) with the **Path Quality** (derived from the consolidated feedback and applying a time-decay factor to older data).
    - Returns the list of paths ordered by this score.
- `func_maintenance_job`
  - **Trigger:** Scheduled Cron Job (e.g., Nightly).
  - **Responsibility:** Periodically applies the **Time-Decay Function** to the database. It archives or lowers the weight of very old reports/feedbacks to ensure that the `func_path_search` always prioritizes fresh information without needing to process the entire history during every query.
- `func_weather_enricher`
  - **Trigger:** Database event (INSERT on `Activities`).
  - **Responsibility:** It securely calls external Meteorological APIs (using backend-only API keys) to fetch weather conditions for the specific time and location of the activity, enriching the path metadata.
- `func_path_deduplication`
  - **Trigger:** Direct Client Request (RPC via check_path_duplicate)
  - **Responsibility:** It implements the **Geometric Integrity Control Logic.** Before a new manual path is saved to the database, this function receives the geometry drawn by the user and executes a high-performance spatial query to verify the existence of already present public paths that are geometrically identical or extremely similar. It returns the validation result to the client, preventing the creation of redundant duplicates and ensuring path uniqueness within the system.

Copyright © 2025, Luca Suzani, Alberto Spezzapria, Alessio Pesavento – All rights reserved 

## 2.3 Deployment view

The deployment architecture of **Best Bike Paths** follows a distributed, client-server model relying on a **Serverless Infrastructure**. This approach minimizes operational overhead, as the backend resources (Database, Auth, Compute) are provisioned and scaled automatically by the Cloud Provider.

The system is deployed across three primary types of nodes:

1. **Mobile User Device (Android/iOS):**
   - **Hardware:** Personal Smartphone.
   - **Execution Environment:** Native OS hosting the **Capacitor Runtime** (WebView).
   - **Artifacts:** The *Hybrid Mobile Application* package (`.apk` / `.ipa`). This artifact contains the compiled Javascript/React bundle and the native plugins required to access hardware sensors (GPS, Accelerometer). It includes an embedded SQLite database for local persistence.
2. **Web User Station (Desktop/Laptop):**
   - **Hardware:** Personal Computer.
   - **Execution Environment:** Modern Web Browser (Chrome, Firefox, Safari).
   - **Artifacts:** The *Progressive Web Application (PWA)* assets (HTML, CSS, JS bundles) cached by the Service Worker.
3. **Managed Cloud Platform (BaaS Provider):**
   - **Hardware:** Distributed Virtualized Clusters (managed by Supabase/AWS).
   - **Execution Environment:** Containerized Database Engine and V8 Isolate runtime for Edge Functions.
   - **Artifacts:**
     - *PostgreSQL Instance:* Stores data and RLS policies.
     - *Auth Service Instance:* Handles identity management.
     - *Edge Functions:* Deployed serverless scripts (`path_merging`, `score_calc`) executed on demand.

### 2.3.1 Network Communication

Communication between the Client Nodes and the Cloud Platform occurs exclusively over secure standard protocols:

- **HTTPS (TLS 1.3):** For all REST API calls (Database CRUD, RPC functions).
- **WSS (Secure WebSockets):** For real-time event subscriptions (e.g., live feedback updates).

Copyright © 2025, Luca Suzani, Alberto Spezzapria, Alessio Pesavento – All rights reserved 

## 2.4 Runtime view

### [UC1] User Registration Sequence Diagram

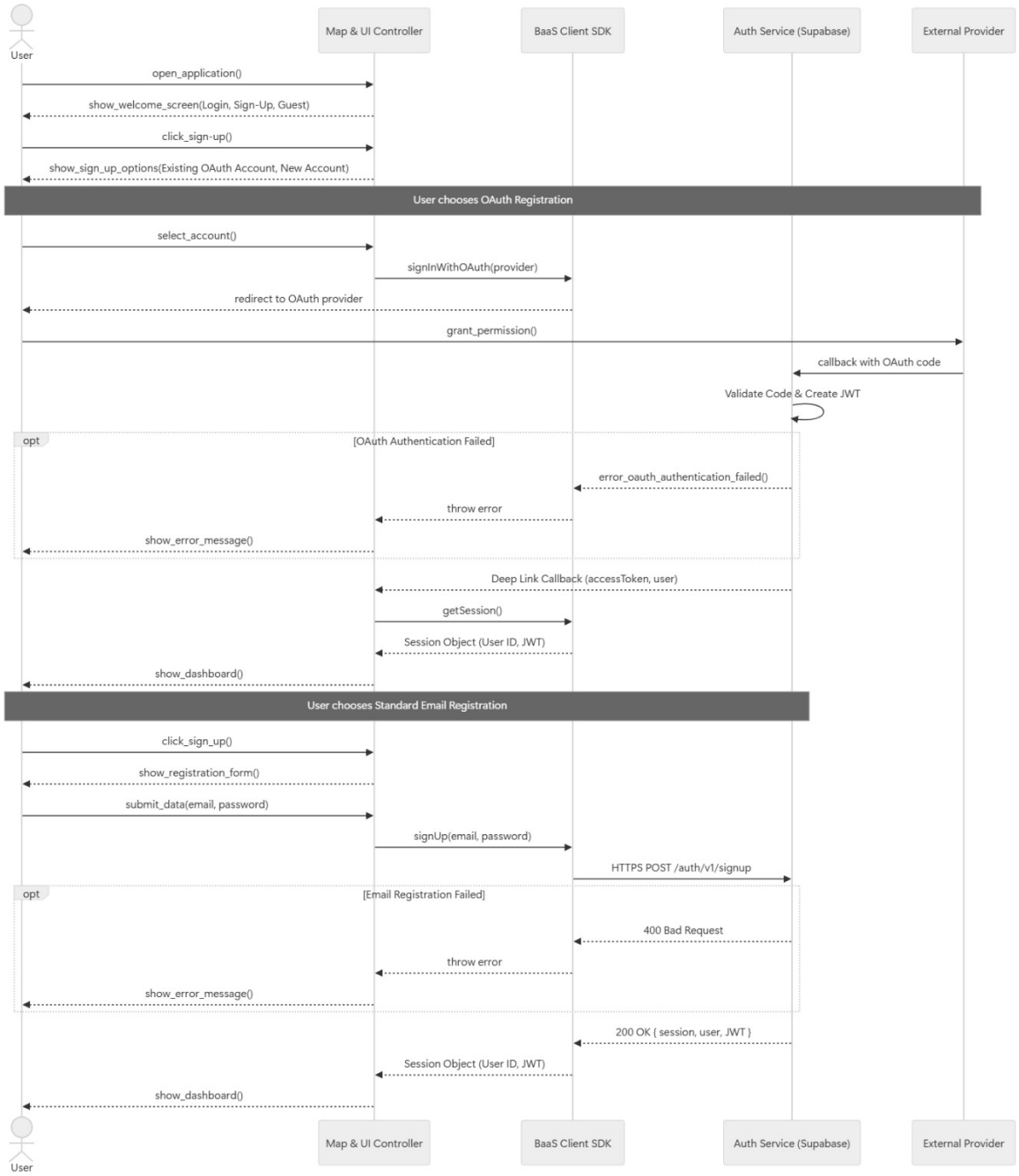



Figure 2.1: [UC1] User Registration Sequence Diagram

The diagram illustrates the registration process managed by the Auth Service. In the OAuth scenario, the flow delegates authentication to the external provider via redirect and callback (Deep Link). In the Email scenario, the SDK communicates directly with the Supabase API. In both cases, the final result is the acquisition of a valid JWT (JSON Web Token), which is essential for ensuring secure access to the database via RLS policies in subsequent interactions.

Copyright © 2025, Luca Suzani, Alberto Spezzapria, Alessio Pesavento – All rights reserved 

**[UC2] System Access Sequence Diagram**

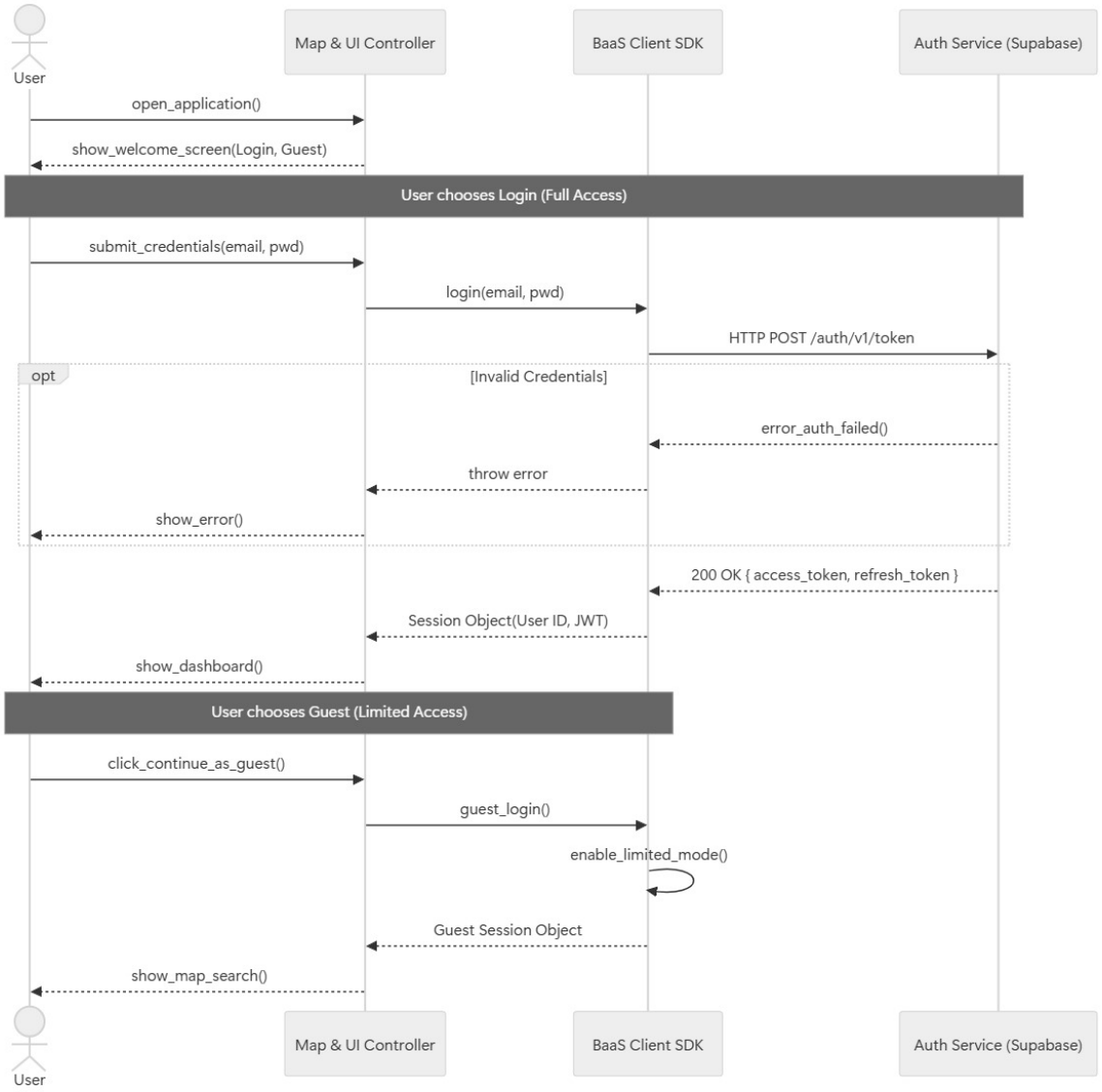



Figure 2.2: [UC2] System Access Sequence Diagram

This diagram depicts the session negotiation process. For registered users, the BaaS Client SDK exchanges credentials for a persistent session token. For Guest users, the SDK initializes a local session in "Limited Mode," which disables write calls to the database and restricts access exclusively to public search APIs.

Copyright © 2025, Luca Suzani, Alberto Spezzapria, Alessio Pesavento – All rights reserved 

### [UC3] Activity Recording Sequence Diagram

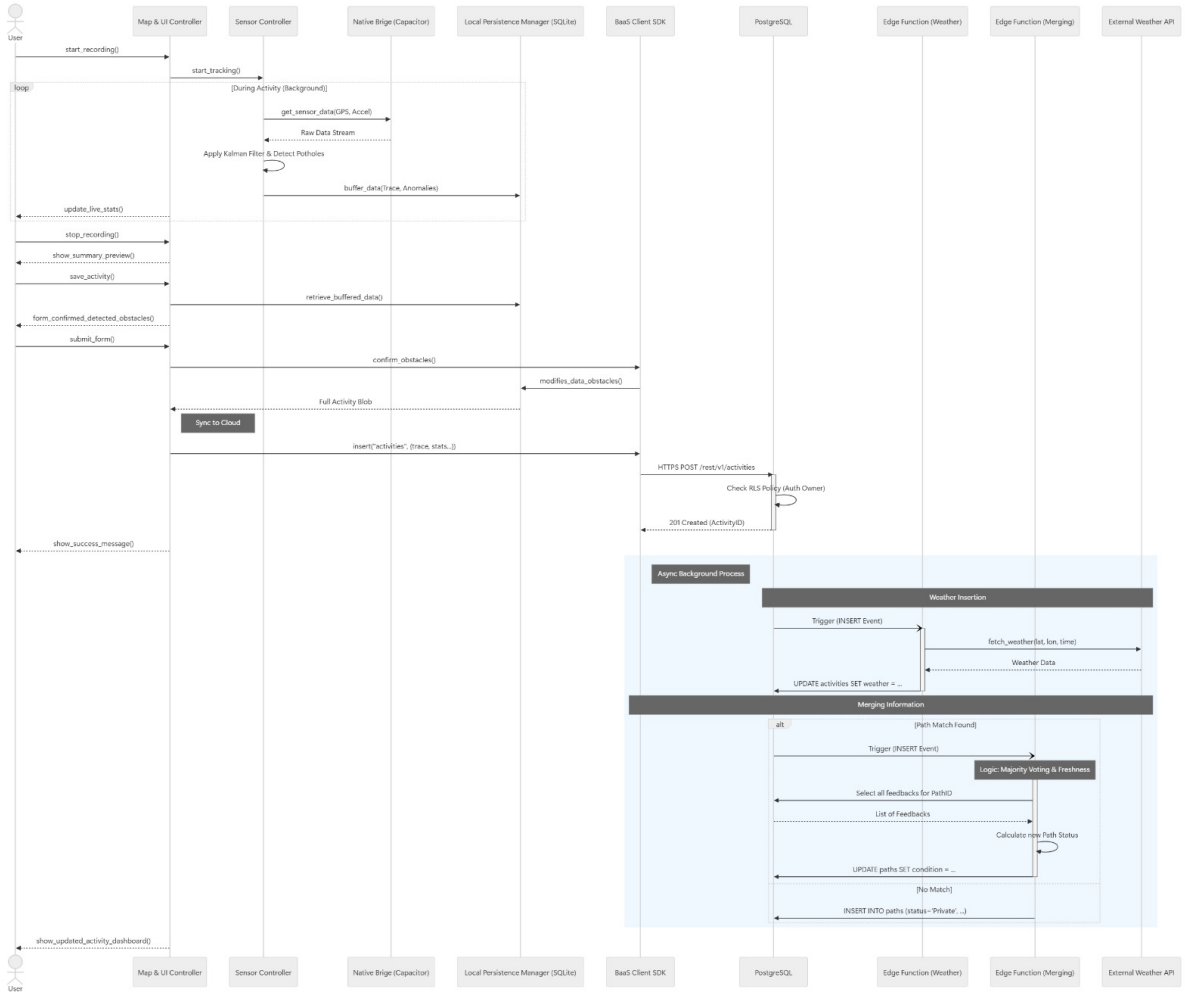



Figure 2.3: [UC3] Activity Recording Sequence Diagram

The diagram highlights the Thick Client and Offline-First architecture. Data acquisition is performed via the Native Bridge, filtered, and saved locally on SQLite. Synchronization with the Cloud triggers asynchronous processes (Serverless): meteorological enrichment and the Merging logic. Note that the trace comparison logic occurs entirely server-side (Edge Function) to avoid burdening the user device and to guarantee data consistency.

Copyright © 2025, Luca Suzani, Alberto Spezzapria, Alessio Pesavento – All rights reserved 

### [UC4] Creation of a manual path Sequence Diagram

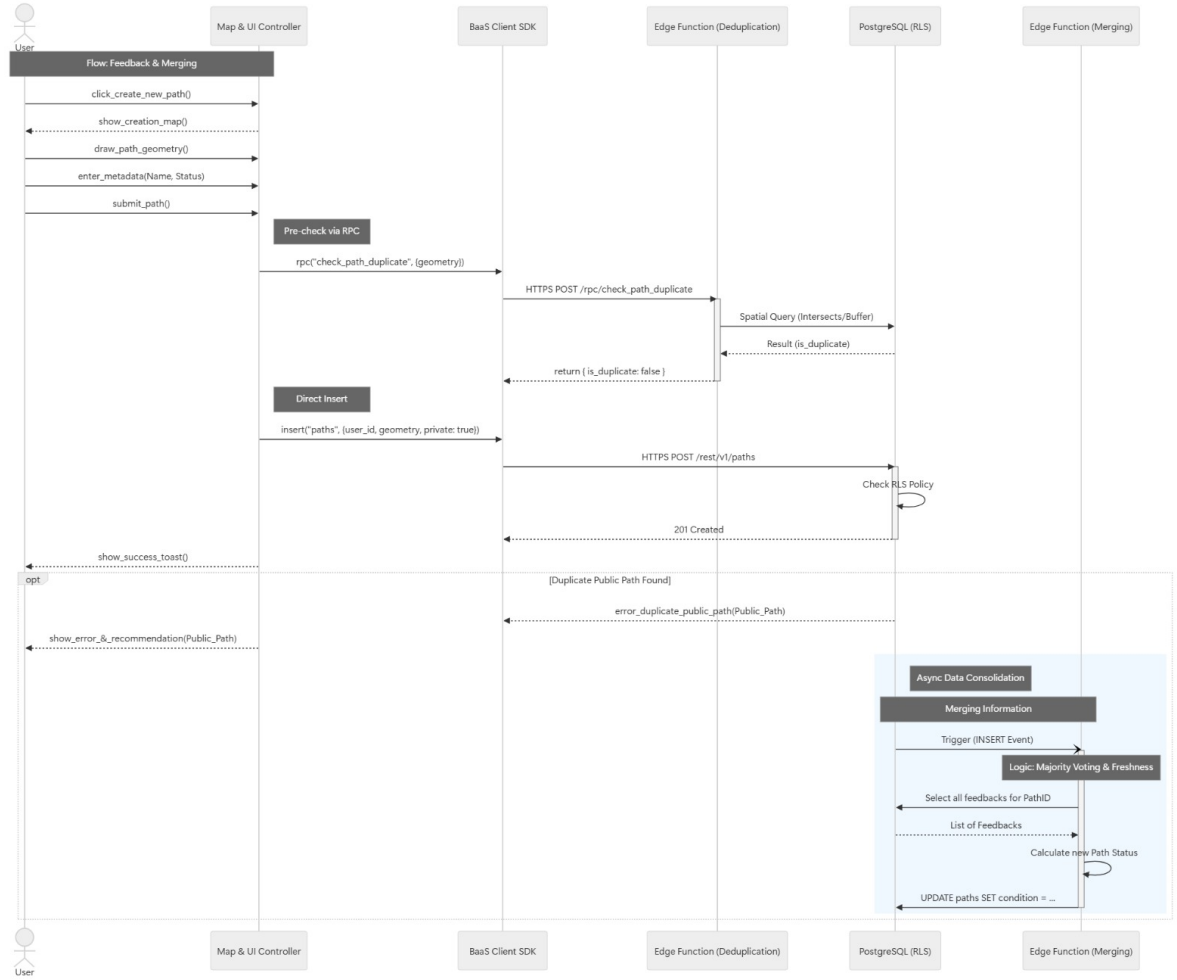



Figure 2.4: [UC4] Creation of a manual path Sequence Diagram

This scenario illustrates the hybrid interaction between direct DB calls and server-side logic. Before inserting the path, the client invokes a Remote Procedure Call (RPC) on a specific Edge Function (Deduplication) to verify the existence of duplicate paths via complex spatial queries (PostGIS). If the check passes, the client proceeds with direct insertion into the database; otherwise, it merges the information entered by the user into the existing public path that corresponds to the inserted geometry.

Copyright © 2025, Luca Suzani, Alberto Spezzapria, Alessio Pesavento – All rights reserved 

**[UC5] Publication of a path Sequence Diagram**

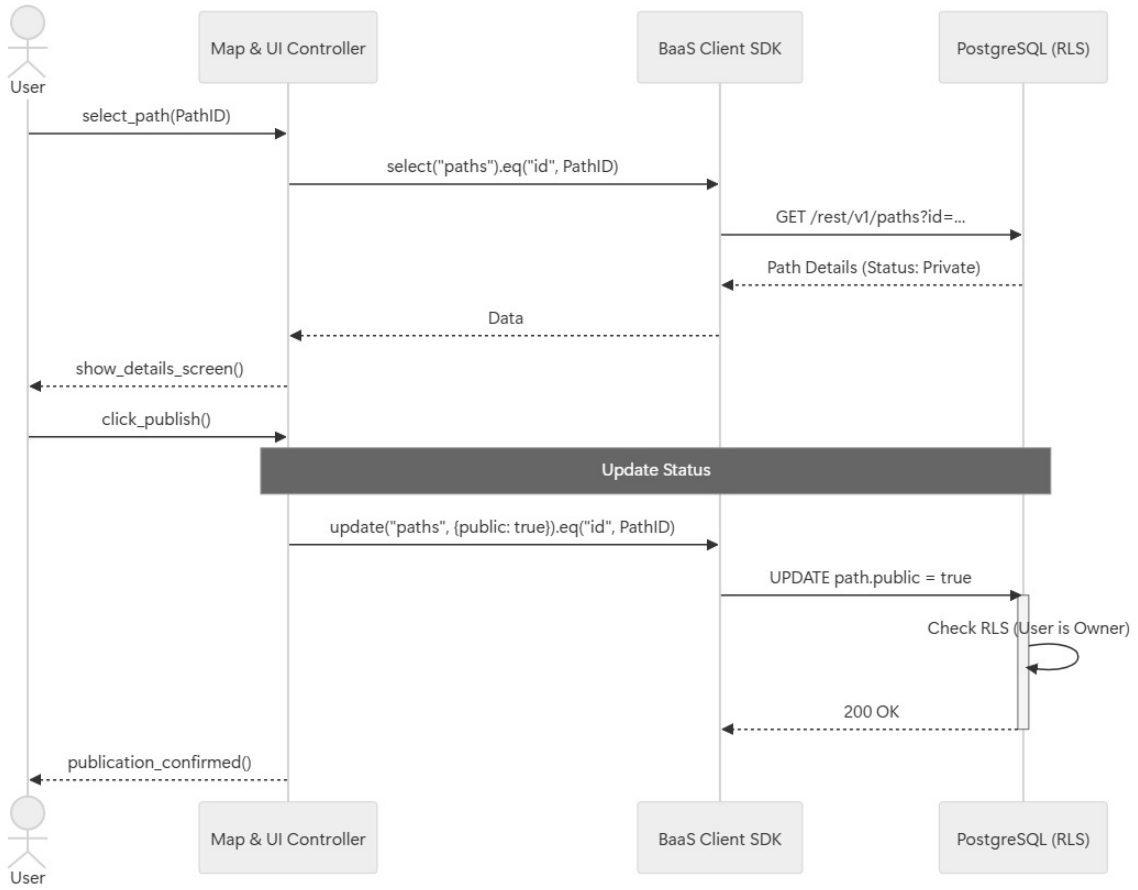



Figure 2.5: [UC5] Publication of a path Sequence Diagram

This represents the state transition of a path from Private to Public. The operation is a direct UPDATE call via the SDK. Security is guaranteed by the PostgreSQL database engine, which, through RLS policies, verifies that the request author corresponds to the record owner before applying the modification.

Copyright © 2025, Luca Suzani, Alberto Spezzapria, Alessio Pesavento – All rights reserved 

**[UC6] Path Feedback reporting Sequence Diagram**

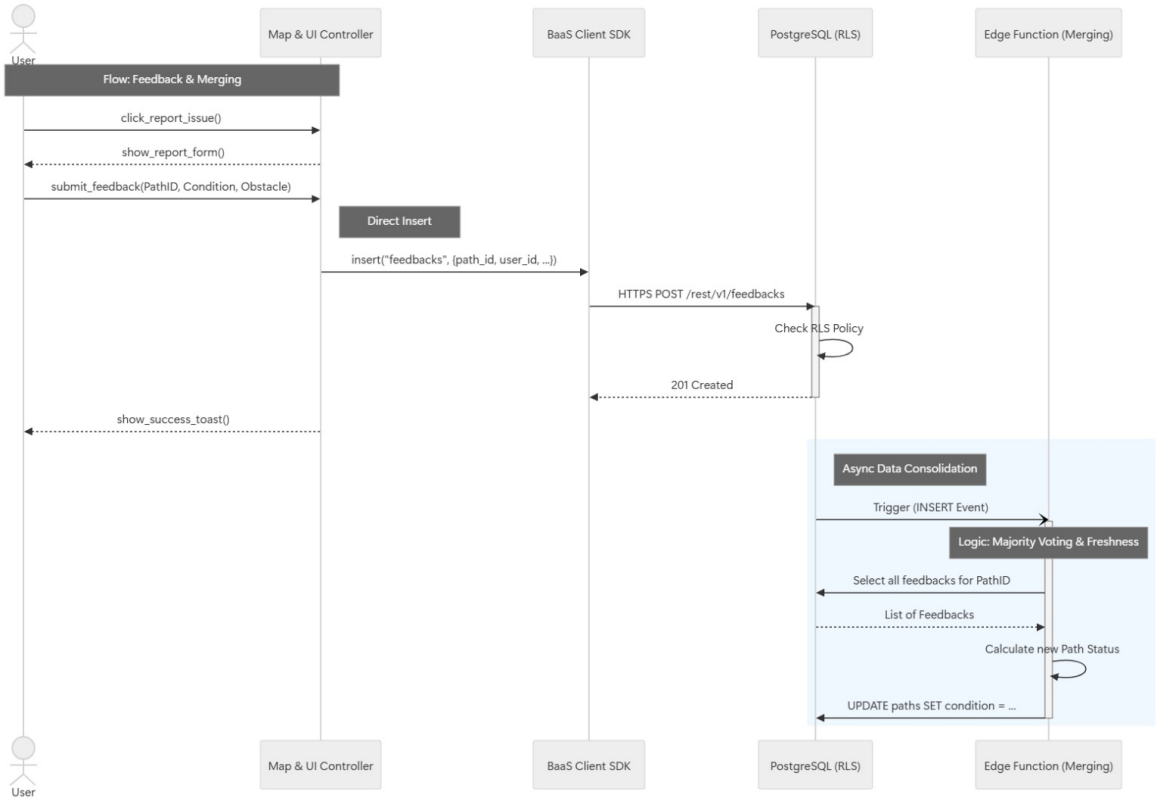



Figure 2.6: [UC6] Path Feedback reporting Sequence Diagram

The diagram demonstrates the Event-Driven architecture. The client handles only the rapid insertion of the feedback to ensure user responsiveness. The database emits an internal event (Trigger) that asynchronously activates the Merging Edge Function. This function recalculates the path status by aggregating existing feedbacks according to the "Majority Voting" algorithm defined in the RASD.

Copyright © 2025, Luca Suzani, Alberto Spezzapria, Alessio Pesavento – All rights reserved 

**[UC7] Search for a path Sequence Diagram**

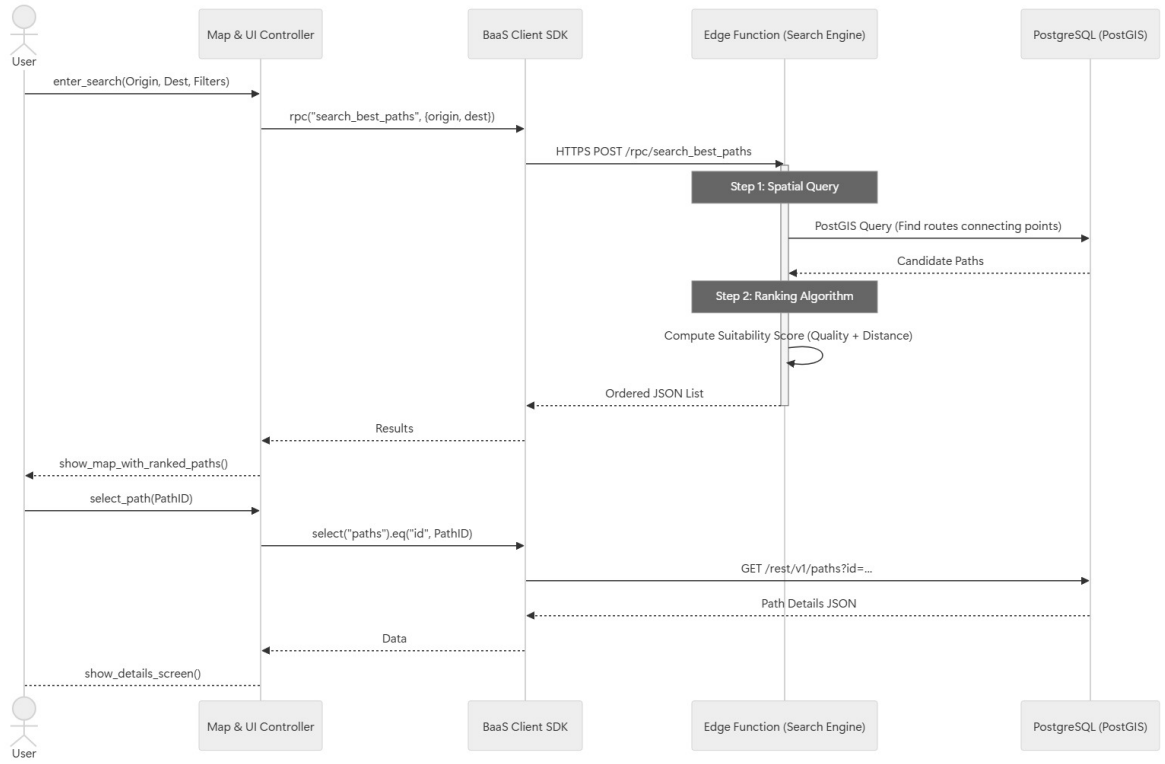



Figure 2.7: [UC7] Search for a path Sequence Diagram

Since the ranking algorithm requires complex calculations (combining geometric distance and quality score), the search is not a simple DB query but is encapsulated in an RPC (Remote Procedure Call). The Edge Function executes the spatial query on PostGIS and sorts the results in memory before returning them to the client, optimizing performance and reducing data traffic.

## 2.5 Component interfaces

This section specifies the APIs provided and required by the major components identified in the Component View. The interfaces are categorized into **Hardware Abstraction Interfaces** (internal to the Hybrid Client) and **Network Interfaces** (between Client and Cloud).

### 2.5.1 Hardware Abstraction Interfaces

These interfaces are exposed by the **Native Bridge (Capacitor)** and consumed by the **Sensor Controller** component within the web runtime. They allow the TypeScript application logic to access device sensors.

**Interface: IGeolocationProvider**

- **Description:** Provides access to the device's GPS hardware.
- **Provided By:** Native Bridge (`@capacitor/geolocation`)
- **Used By:** Sensor Controller

Copyright © 2025, Luca Suzani, Alberto Spezzapria, Alessio Pesavento – All rights reserved 

| Method Signature | Input Parameters | Return Value | Description |
|---|---|---|---|
| `getCurrentPosition()` | `options: GeoOptions` | `Promise<Position>` | Returns the single current coordinate (Lat, Long, Altitude). |
| `watchPosition()` | `options: GeoOptions, callback: (pos) => void` | `CallbackID` | Starts a continuous background location tracking session. The callback is invoked every time the OS receives a location update. |
| `clearWatch()` | `id: CallbackID` | `void` | Stops the tracking session to save battery. |

Table 2.1: IGeolocationProvider Interface

**Interface: IMotionSensor**

- **Description:** Provides high-frequency access to the accelerometer for pothole detection.
- **Provided By:** Native Bridge (`@capacitor/motion`)
- **Used By:** Sensor Controller (Inertial Analyzer)

| Method Signature | Input Parameters | Return Value | Description |
|---|---|---|---|
| `addListener()` | `eventName: 'accel', callback: (event) => void` | `PluginListenerHandle` | Subscribes to the raw accelerometer stream. The event contains $x, y, z$ acceleration vectors used to detect vertical shocks. |
| `removeAllListeners()` | `void` | `void` | Unsubscribes from the sensor stream. |

Table 2.2: IMotionSensor Interface

### 2.5.2 Cloud Network Interfaces (BaaS API)

These interfaces represent the communication contract between the **Hybrid Client** and the **Serverless Cloud**. Since the architecture uses a BaaS, these are primarily RESTful endpoints automatically generated by Supabase from the database schema, plus RPC (Remote Procedure Calls) for Edge Functions.

**Interface: IAuthService**

- **Description:** Manages user sessions.
- **Protocol:** HTTPS / REST
- **Provided By:** Auth Service (Supabase GoTrue)

Copyright © 2025, Luca Suzani, Alberto Spezzapria, Alessio Pesavento – All rights reserved 

| Endpoint / Method | Input Payload | Response | Description |
|---|---|---|---|
| `POST /auth/v1/signup` | `{ email, password }` | `{ user, session, jwt }` | Registers a new user and returns the access token. |
| `POST /auth/v1/token` | `{ email, password, grant_type }` | `{ access_token, refresh_token }` | Authenticates a user (Login). |
| `POST /auth/v1/logout` | Header: `Bearer <token>` | `204 No Content` | Invalidates the current session. |

Table 2.3: IAuthService Interface

**Interface: ISpatialDataStore**

- **Description:** Provides CRUD access to geospatial entities. Access is filtered by RLS policies.
- **Protocol:** HTTPS / REST
- **Provided By:** Spatial Database (PostgREST)

| Endpoint / Method | Input Payload | Response | Description |
|---|---|---|---|
| `GET /rest/v1/paths` | Query Params: `select=*, geometry=...` | `JSON Array` | Retrieves public paths. Supports spatial filtering (e.g., retrieving paths within a map viewport). |
| `POST /rest/v1/activities` | `{ user_id, geometry_linestring, ... }` | `201 Created` | Uploads a recorded activity trace. This action triggers the `func_path_merging` Edge Function asynchronously. |
| `POST /rest/v1/feedbacks` | `{ path_id, rating, comment }` | `201 Created` | Submits a user report for a specific path. |

Table 2.4: ISpatialDataStore Interface

**Interface: ISearchEngine (RPC)**

- **Description:** Exposes the complex path ranking logic as a remote function call.
- **Protocol:** HTTPS / RPC
- **Provided By:** Edge Functions

Copyright © 2025, Luca Suzani, Alberto Spezzapria, Alessio Pesavento – All rights reserved 

| Function Name | Input Parameters | Return Value | Description |
|---|---|---|---|
| `rpc/search_best_paths` | `{ origin_lat, origin_lon, dest_lat, dest_lon }` | `JSON Array <Path>` | Invokes the routing algorithm. Returns a list of paths ordered by the dynamic **Suitability Score** (calculated based on distance and road quality). |

Table 2.5: ISearchEngine Interface

### 2.5.3 External Interfaces

**Interface: IWeatherProvider**

- **Description:** External API used to enrich path data with meteorological conditions.
- **Provided By:** Third-party provider (e.g., OpenWeatherMap)
- **Used By:** `func_weather_enricher` (Edge Function)

| Endpoint | Method | Params | Description |
|---|---|---|---|
| `/data/2.5/weather` | `GET` | `lat, lon, appid` | Fetches current weather data (Temperature, Rain volume, Wind) for a specific coordinate. |

Table 2.6: IWeatherProvider Interface

## 2.6 Selected architectural styles and patterns

### 2.6.1 Architectural Styles

- **Serverless / Backend-as-a-Service (BaaS)**
  - **Description:** The system does not rely on a traditional, self-managed monolithic application server. Instead, backend responsibilities, including database management, authentication, and file storage, are delegated to a managed cloud provider (Supabase). Application logic is executed via stateless containers (Edge Functions) that spin up only when triggered.
  - **Rationale:** This style was chosen to maximize scalability and minimize operational overhead. Since the "Best Bike Paths" system must handle potential spikes in usage without manual provisioning, a serverless infrastructure ensures automatic scaling. Furthermore, it allows the development team to focus entirely on the complex domain logic (merging and ranking) rather than server maintenance and patching.
- **Event-Driven Architecture**
  - **Description:** The interaction between data ingestion and data processing is decoupled. When a client uploads a new activity, the system does not process it synchronously. Instead, the database emits an event (webhook), which asynchronously triggers the specific Edge Function responsible for the merging algorithm.

Copyright © 2025, Luca Suzani, Alberto Spezzapria, Alessio Pesavento – All rights reserved 

  - **Rationale:** The Path Merging process involves complex geospatial calculations (intersections, buffering) that are computationally expensive. Processing them synchronously would block the client UI and lead to connection timeouts. An event-driven approach ensures the user interface remains responsive (Latency Hiding) while heavy tasks are processed in the background.

- **Thick Client**

  - **Description:** A significant portion of the application logic resides on the client device rather than the server. The Hybrid Client is responsible for sensor data acquisition, noise filtering (Kalman Filter), data validation, and local buffering.
  - **Rationale:** This style is strictly required to satisfy the reliability constraints defined in the RASD. Cyclists often traverse areas with no network coverage; the client must be able to function autonomously, recording and validating data locally, and syncing only when connectivity is restored.

### 2.6.2 Design Patterns

- **Bridge Pattern (via Capacitor)**

  - **Description:** The application utilizes a native bridge to decouple the high-level application logic (written in TypeScript/React) from the low-level platform implementation (Android/iOS). The web code calls a uniform JavaScript API, and the Bridge translates these calls into native OS commands.
  - **Rationale:** This pattern enables cross-platform accessibility while satisfying the need for hardware-specific features. Specifically, Capacitor native plugins allows the web-based application to access the accelerometer and background geolocation services, which are otherwise inaccessible or limited in a standard browser environment.

- **Observer Pattern**

  - **Description:** The client application subscribes to real-time changes on specific database tables (e.g., `Feedbacks`) using Secure WebSockets (WSS). When the data changes on the server, the new state is automatically pushed to all listening clients.
  - **Rationale:** This is essential for the safety-critical feature of reporting road hazards. If a user reports a danger, other nearby cyclists must be notified immediately to update their map view, without needing to manually refresh the application.

- **Row Level Security (RLS)**

  - **Description:** Access control logic is implemented directly within the database engine rather than in the application code. Each database query is evaluated against a set of SQL policies that check the user's authentication token against the data being accessed.
  - **Rationale:** Since the architecture allows clients to communicate directly with the database (skipping the middleware layer), RLS is the only mechanism to guarantee dData isolation. It mathematically ensures that a user can never read or modify another user's private data, regardless of any potential bug in the client code.

Copyright © 2025, Luca Suzani, Alberto Spezzapria, Alessio Pesavento – All rights reserved 

## 2.7 Other design decisions

### 2.7.1 Adoption of Hybrid Mobile Architecture

- **Description:** The development strategy involves building a single web-based codebase wrapped in a native container (Capacitor), rather than developing two separate native applications (Kotlin for Android, Swift for iOS) or a pure Progressive Web App (PWA).
- **Rationale:** A pure PWA was discarded because it cannot reliably access high-frequency motion sensors in the background, which is critical for the "Road Quality Detection" requirement. Native development was discarded due to the high effort required for a small team. The Hybrid approach offers the necessary hardware access with a unified codebase, significantly reducing development and testing time.

### 2.7.2 Usage of PostgreSQL with PostGIS

- **Description:** The system utilizes a Relational Database Management System (RDBMS) extended with the PostGIS spatial engine to store and query all geospatial data (Points, Linestrings, Polygons).
- **Rationale:** Generic databases are insufficient for the mathematical complexity of this project. PostGIS provides native, highly optimized functions for geospatial conflation and Nearest-Neighbor searches. Implementing these geometric algorithms from scratch in code would be error-prone and inefficient.

### 2.7.3 Centralized Logic for Merging and Ranking

- **Description:** The core algorithms that determine the shape of public paths (Merging) and their value (Ranking) are executed exclusively on the Cloud Tier via Edge Functions, never on the client.
- **Rationale:** This decision ensures a **Single Source of Truth**. The merging algorithm requires a complete view of all existing paths in an area to make a decision, which a single client does not possess. Furthermore, calculating the Ranking on the server prevents malicious users from tampering with the client code to artificially inflate the score of their preferred paths.

### 2.7.4 Direct-to-Database Communication Pattern

- **Description:** The architecture bypasses the traditional "API Gateway / Controller" layer for standard operations. Clients perform CRUD (Create, Read, Update, Delete) operations directly against the database's REST API.
- **Rationale:** The majority of user interactions (e.g., loading a profile, saving a track) are standard data operations that do not require complex business logic. Implementing a dedicated middle-layer for these simple tasks would introduce unnecessary latency and code maintenance ("Boilerplate"). Security is delegated to RLS (as described in 2.6.2), making the middle layer redundant for these tasks.

Copyright © 2025, Luca Suzani, Alberto Spezzapria, Alessio Pesavento – All rights reserved 

# Chapter 3

# User Interface Design

This section outlines the main user interface views and the rationale behind their layout. Each screen is described with its **primary goals**, **key interactions**, and **visual hierarchy** choices to ensure quick comprehension, consistency across flows, and usability in both **desktop** and **mobile** contexts.

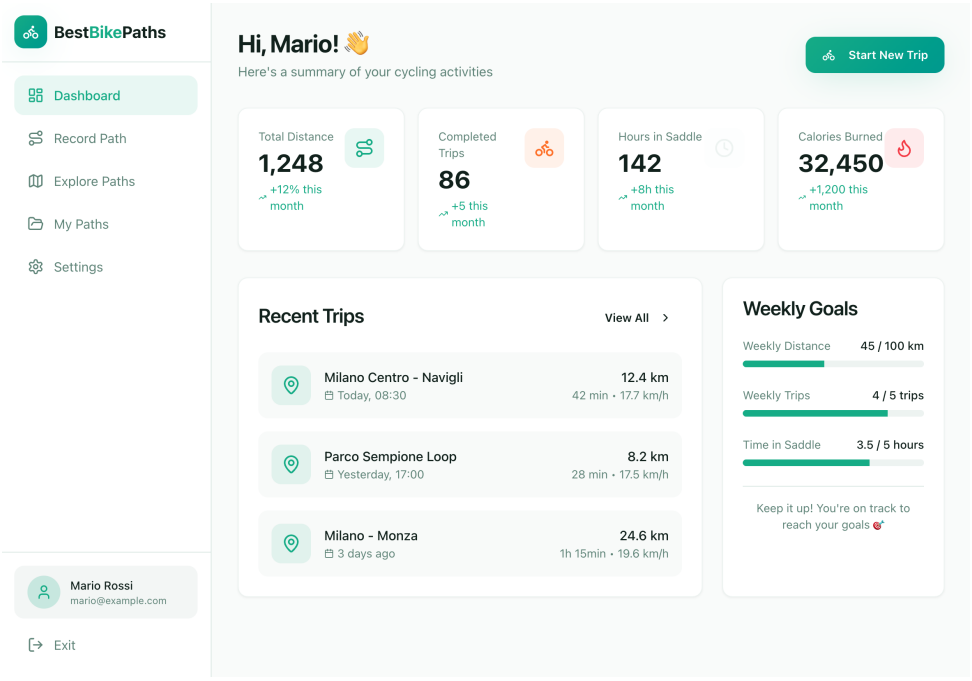



Figure 3.1: Main dashboard summarizing service status, recent notifications, and quick actions.

The **dashboard** provides an immediate summary of the most relevant information: **active services status**, **recent notifications**, **shortcuts** to the most frequent actions, and compact **trend charts**. Content is organized in **cards** to allow fast scanning and **responsive** flow; each card highlights the primary **KPI** or status and, when relevant, a **trend indicator**. A **left navigation bar** leads to the operational sections, while the **header** shows the name of the user and a button to **record a new activity**.

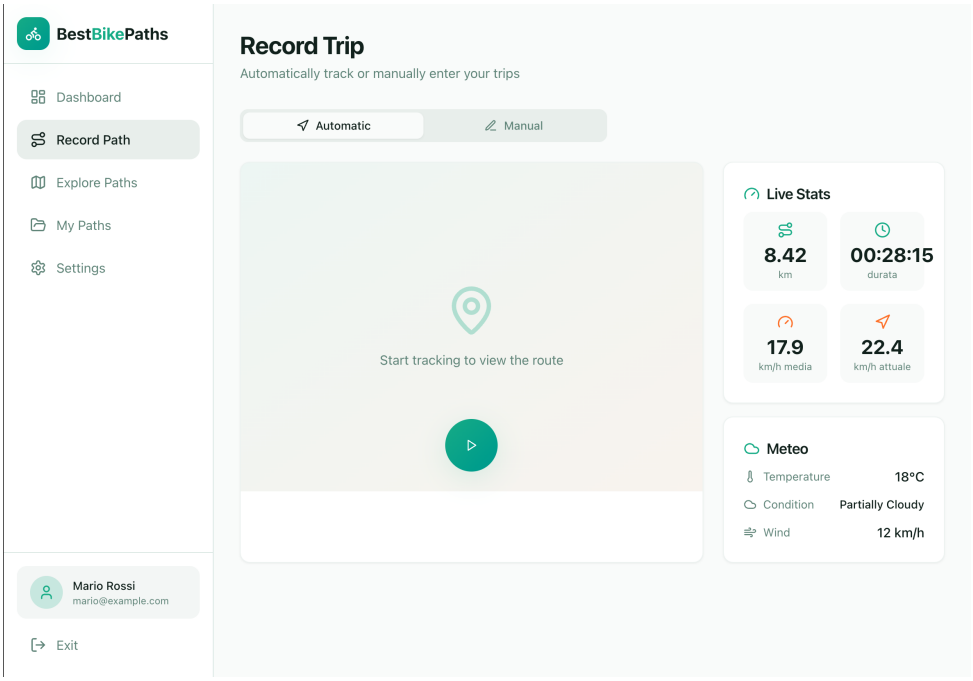



Figure 3.2: Trip recording interface with live metrics and waypoint controls.

The recording screen foregrounds the **start/stop control** and **live metrics** (**distance**, **duration**, **pace**). Secondary actions allow adding **waypoints** or **notes**; the layout keeps high **contrast** for outdoor readability.

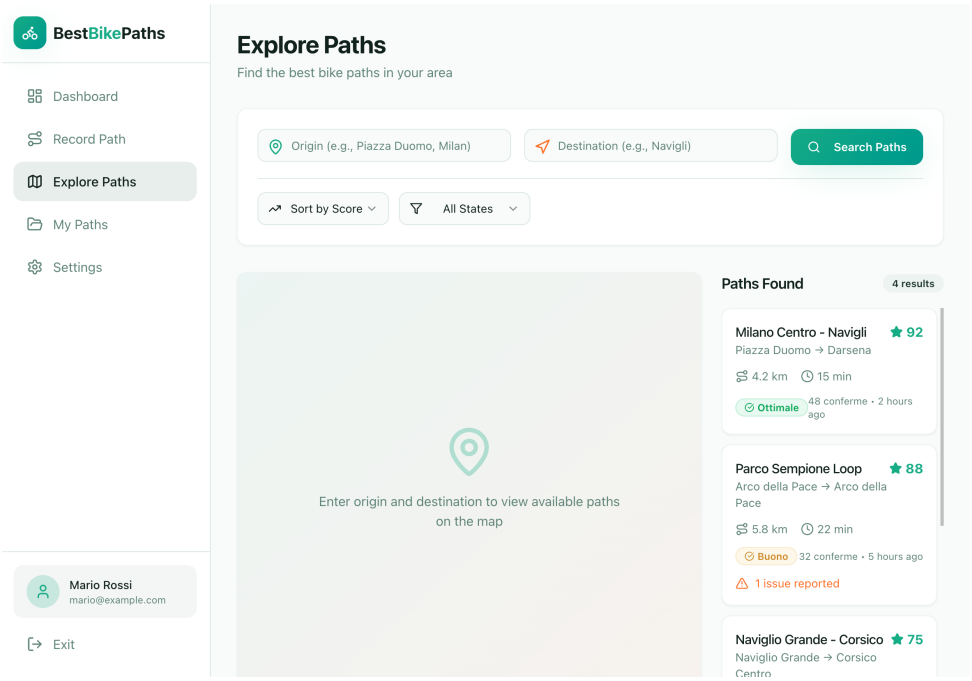



Figure 3.3: Explore curated and suggested routes with filters and key stats.

The explore view offers **curated and suggested routes** with **filters** for **difficulty**, **duration**, and **location**. Cards show a brief **description**, **tags**, and primary **stats** to help selection at a glance.

Copyright © 2025, Luca Suzani, Alberto Spezzapria, Alessio Pesavento – All rights reserved 

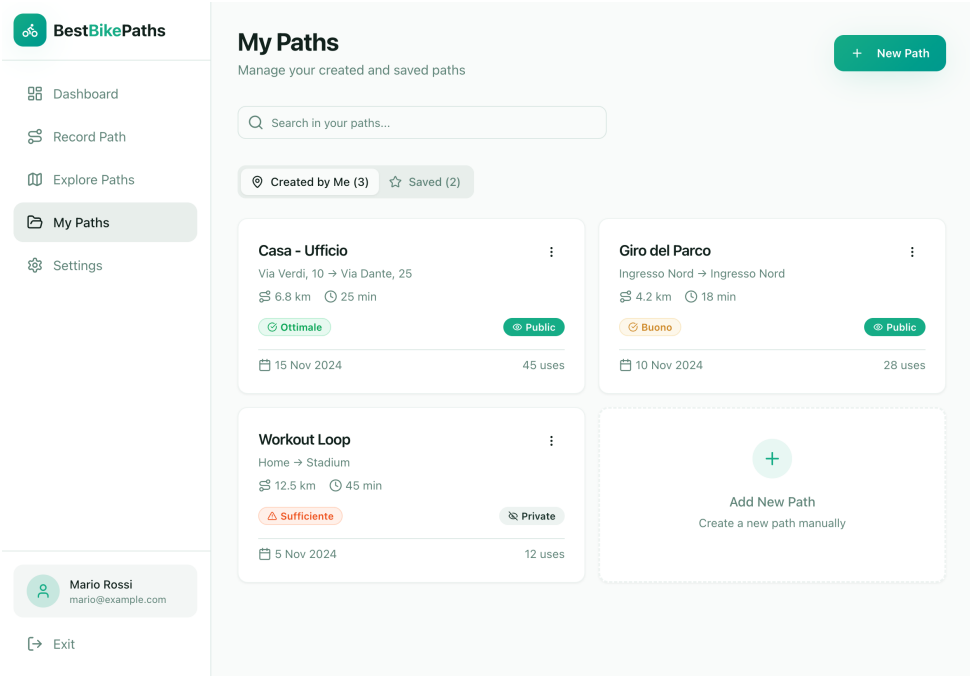



Figure 3.4: List of saved routes with quick access to details and navigation.

This screen lists the user **saved routes** with key **metadata** (**name**, **distance**, **duration**, **last update**) and quick entry points to view **details** or start **navigation**. Rows are designed for scannability and expose **status badges** for **drafts** or **shared items**.

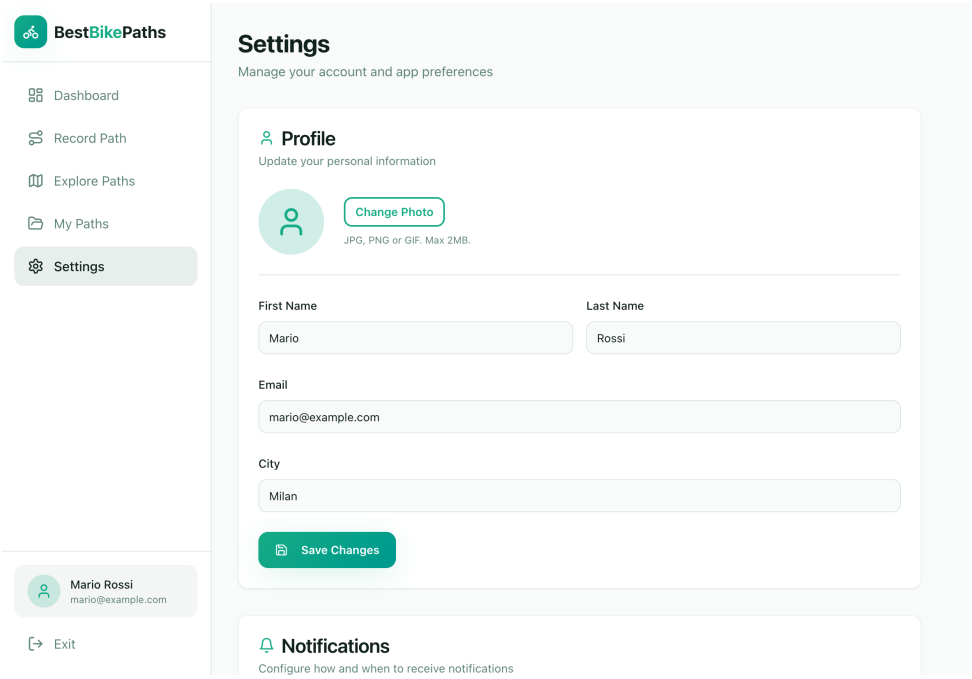



Figure 3.5: Settings grouped by account, privacy, and notifications.

Settings centralize **account**, **privacy**, and **notification preferences**. **Destructive actions** are visually separated.

Copyright © 2025, Luca Suzani, Alberto Spezzapria, Alessio Pesavento – All rights reserved 

# Chapter 4

# Requirements Traceability

## 4.1 Requirements Traceability

The following tables map the functional requirements identified in the RASD to the architectural components defined in the Design Document. This mapping ensures that every requirement is addressed by specific modules within the system.

### 4.1.1 User Account Management

| | |
|---|---|
| **Requirements** | **R1:** The platform allows users to create an account by providing the required personal information.<br>**R2:** The platform allows registered users to log in and access the system.<br>**R3:** The platform allows registered users to recover the access credentials.<br>**R4:** The platform allows guest users to access only the path search functionality. |
| **Components** | • Hybrid Client App (UI)<br>• BaaS Client SDK<br>• Auth Service (Identity Provider)<br>• Spatial Database (Users Table) |

Table 4.1: Traceability for User Account Management

### 4.1.2 Activity Recording

| | |
|---|---|
| **Requirements** | **R5:** The system shall allow registered users to start recording a new cycling activity.<br>**R7:** The system shall collect raw data from sensors (GPS, accelerometer).<br>**R8:** The system shall analyze speed to verify cycling activity.<br>**R9:** The system shall analyze sensors to detect road anomalies.<br>**R10:** The system shall calculate real-time statistics.<br>**R11:** The system shall allow pausing and resuming. |
| **Components** | • Native Bridge (Capacitor)<br>• Sensor Controller & Filter<br>• Local Persistence Manager<br>• Map & UI Controller |

Table 4.2: Traceability for Activity Recording

### 4.1.3 Path Creation (Automatic)

| | |
|---|---|
| **Requirements** | **R12:** System displays detected obstacles upon stopping activity.<br>**R13:** System requires user validation of obstacles.<br>**R14:** System analyzes GPS trace to match existing paths.<br>**R15:** System associates activity with existing path if matched.<br>**R16:** System updates status/info of existing public path.<br>**R18:** System generates new path if no match is found. |
| **Components** | • Map & UI Controller (Validation Interface)<br>• BaaS Client SDK<br>• Spatial Database<br>• `func_path_merging` (Edge Function)<br>• `func_weather_enricher` (Edge Function) |

Table 4.3: Traceability for Automatic Path Creation

### 4.1.4 Path Creation (Manual)

| | |
|---|---|
| **Requirements** | **R20:** User creates path manually by selecting streets on map.<br>**R21:** User manually inputs status and obstacles.<br>**R22:** User assigns name and stores in "My Paths". |
| **Components** | • Map & UI Controller<br>• BaaS Client SDK<br>• Spatial Database |

Table 4.4: Traceability for Manual Path Creation

Copyright © 2025, Luca Suzani, Alberto Spezzapria, Alessio Pesavento – All rights reserved 

### 4.1.5 Search and Ranking

| | |
|---|---|
| **Requirements** | **R23:** Users search paths by origin and destination.<br>**R24:** Users filter search results.<br>**R25:** Platform ranks paths based on condition, freshness, and effectiveness. |
| **Components** | • Map & UI Controller<br>• `func_path_search` (Edge Function / RPC)<br>• Spatial Database (PostGIS Indexing) |

Table 4.5: Traceability for Search and Ranking

### 4.1.6 Path Management

| | |
|---|---|
| **Requirements** | **R26:** Platform detects when user follows a public path.<br>**R27:** Users update condition info without performing activity.<br>**R28:** Users edit info of created paths.<br>**R29:** Users publish stored paths to community. |
| **Components** | • Map & UI Controller<br>• BaaS Client SDK<br>• Spatial Database<br>• `func_path_merging` (For updates/consensus) |

Table 4.6: Traceability for Path Management

Copyright © 2025, Luca Suzani, Alberto Spezzapria, Alessio Pesavento – All rights reserved 

# Chapter 5

# Implementation, Integration and Test Plan

## 5.1 Implementation, Integration & Testing

### 5.1.1 Overview

The implementation strategy for the Best Bike Paths (BBP) system is driven by the need to combine the flexibility of web development with the performance of native hardware. The chosen architecture—Hybrid Mobile (Capacitor) supported by a Serverless Backend (Supabase + Deno)—addresses two critical requirements:

- **Maximizing Data Consistency:** The pervasive use of TypeScript, from the database (via generated definitions) to the mobile client, ensures a "Type-Safe" ecosystem, drastically reducing integration bugs.
- **Optimizing Resource Usage:** By delegating spatial computational complexity to PostGIS and business logic to the Edge, the mobile client remains lightweight, preserving the user device's battery life.

### 5.1.2 Implementation Plan

The development process follows a "Data-Centric Bottom-Up" approach. Instead of starting with the user interface, implementation begins with a rigorous definition of the data model and security rules, progressively building the upper layers. This ensures that every component, once integrated, rests on a stable foundation.

#### Phase 1: Data Integrity & Security Layer

The primary goal is to establish a reliable "Single Source of Truth." This involves not just creating tables, but defining the data contract that will govern the entire system.

- **Geospatial Persistence Engine:** The adoption of PostgreSQL with PostGIS is not merely for storage, but to serve as the primary calculation engine. Path trajectories are modeled as `GEOGRAPHY` types to ensure precise geodetic calculations (independent of map distortion), which are essential for the proximity queries required by functional requirements.
- **Automated Contract Generation:** To prevent misalignment between backend and frontend, a pipeline is implemented to automatically reflect the database schema into static TypeScript interfaces. This mechanism enforces that any change to the data structure (e.g., adding a "slope" field) is immediately propagated to the code, causing preemptive compilation errors rather than runtime failures.

- **Authorization at Storage Level:** Security is implemented directly on the data via Row Level Security (RLS). This "Defense in Depth" approach ensures that access rules (e.g., "only the author sees private paths") are applied universally, regardless of whether the request originates from the API, the admin dashboard, or a future client.

**Phase 2: Distributed Business Logic (Edge Layer)**

Application logic is developed as an intermediate layer of serverless functions (Deno), acting as intelligent glue between the client and the database.

- **Compute Offloading Strategy:** Heavy operations are strategically distributed. The Database handles spatial complexity (intersections, distance calculations) via optimized RPCs. The Edge Function handles algorithmic logic (e.g., path ranking based on dynamic weights) and orchestration. This separation prevents complex calculations from blocking the main database thread or draining the smartphone's CPU.
- **Validation Gatekeeping:** Before any data reaches the database, it is intercepted by a validation layer (based on Zod). This component ensures the structural and semantic sanity of incoming data (e.g., verifying that a GPX track does not have impossible time jumps), protecting system integrity from malformed input or attacks.

**Phase 3: Hybrid Client & Sensor Fusion**

Mobile client development focuses on hardware abstraction and offline resilience, using Capacitor as a bridge to native functionalities.

- **Native Bridge Abstraction:** Capacitor is used to access critical sensors (GPS, Accelerometer), bypassing browser limitations. The goal is to ensure background execution: the system must continue recording high-frequency data even when the device is locked, a feature not reliably available in standard Web Apps.
- **Local-First Architecture:** Given the mobile nature of the application (usage in areas with poor coverage), the client is designed to operate autonomously. A persistent local buffer (via SQLite) is implemented to act as temporary storage.
- **Synchronization Logic:** An intelligent synchronization manager monitors network status. Instead of attempting immediate and fallible uploads, the system accumulates data locally and transmits it in batches to the ingestion Edge Function only when the connection is stable, implementing "retry" logic to ensure no activity is lost.

**Phase 4: Asynchronous Consistency**

The final phase handles processes that do not require real-time interaction, decoupling the user experience from heavy workloads.

- **Event-Driven Updates:** To keep path statistics updated without slowing down feedback submission, an event-driven model is adopted. Inserting a new feedback asynchronously triggers (via Database Webhooks) the recalculation of the path's score. This ensures the user interface remains "snappy" while the system processes data in the background.

Copyright © 2025, Luca Suzani, Alberto Spezzapria, Alessio Pesavento – All rights reserved 

### 5.1.3 Component Integration

Integration in BBP is not a final phase, but a continuous process guaranteed by the architecture itself.

- **Static Contract Verification:** Integration between modules (Mobile Client ↔ Edge Function ↔ Database) is verified statically at compile time. Sharing TypeScript types acts as a formal contract: if the database schema changes, the client will not compile until it is updated to handle the change. This eliminates entire classes of integration bugs.
- **Environment Parity:** To mitigate deployment risks, the local development environment faithfully replicates the production infrastructure (including containers for Edge Functions and PostGIS). This allows developers to test full flows (from simulated sensor to storage) on their own machines before even reaching the staging environment.

### 5.1.4 Testing Strategy

The testing strategy is designed to mitigate risks specific to an application based on geolocation and physical sensors.

#### Logic & Algo Testing (Unit)

The focus is on algorithmic correctness. Since ranking and data fusion logic is isolated in pure TypeScript/Deno functions, it can be tested exhaustively with fast unit tests by mocking external dependencies. This ensures the mathematics behind score calculation is correct before involving the database.

#### Native & Hardware Testing (System)

Traditional tests are insufficient to validate hardware interaction.

- **Energy Profiling:** Instrumental monitoring of energy consumption is performed (via native Android/iOS profilers) to ensure background sensor sampling respects battery constraints (Requirement 3.3).
- **Background Lifecycle Verification:** Field tests are crucial to verify that the operating system does not "kill" the recording process when memory is under pressure or the screen is off for long periods.
- **Network Chaos:** Simulation of adverse network conditions (signal loss, switching between 4G/WiFi) to validate the robustness of the offline synchronization mechanism and prevent data loss.

Copyright © 2025, Luca Suzani, Alberto Spezzapria, Alessio Pesavento – All rights reserved 

# Chapter 6

# Effort Spent

| Chapter | Date | Alessio P. | Alberto S. | Luca S. |
|---|---|---|---|---|
| Introduction | 29/12/2025 | 1h | 1h | - |
| | 31/12/2025 | 1h | 1h | 1h |
| | 05/01/2026 | 1h | 1.5h | 1h |
| | **Total** | **3h** | **3.5h** | **2h** |
| Architecture Design | 30/12/2025 | 1h | 3.5h | - |
| | 31/12/2025 | - | 2h | - |
| | 01/01/2026 | - | 2h | - |
| | 03/01/2026 | - | - | 2.5h |
| | 05/01/2026 | - | - | 2h |
| | 06/01/2026 | - | - | 2h |
| | **Total** | **1h** | **7.5h** | **6.5h** |
| User Interface Design | 05/01/2026 | 4h | - | - |
| | 06/01/2026 | 2h | - | - |
| | **Total** | **6h** | **0h** | **0h** |
| Requirements Traceability | 01/01/2026 | - | 2h | - |
| | **Total** | **0h** | **2h** | **0h** |
| Implementation, Integration and Test Plan | 30/12/2025 | 3h | 0.5h | - |
| | **Total** | **3h** | **0.5h** | **0h** |
| **TOTAL** | | **13h** | **13.5h** | **8.5h** |

Table 6.1: Hours of work spent by each author redacting the DD document